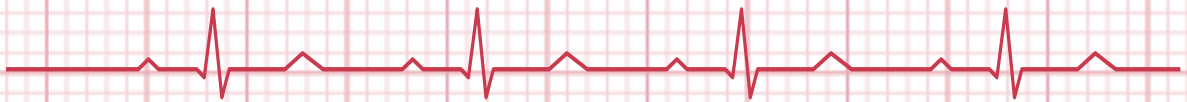


MANUAL DE CLONACIÓN Y DESPLIEGUE · V1.0

CardioAlert

Clonar y desplegar

Guía completa para reproducir el sistema desde cero, con tu propia base de datos, tus propios servidores y tu propia app, y para adaptarlo a otros usos.



REPOSITORIO

github.com/ttziu/cardioalert
(público)

TIEMPO ESTIMADO

2 a 4 horas la primera vez

FECHA

Septiembre de 2026

CONTENIDO

1. Qué es y qué obtienes al clonarlo
2. Arquitectura y dónde se aloja cada parte
3. Requisitos y costos
4. Estructura del repositorio
5. Paso 1 · Clonar el repositorio
6. Paso 2 · Base de datos en Supabase
7. Paso 3 · Modelo de IA en Google Colab
8. Paso 4 · Desplegar la API de IA
9. Paso 5 · Desplegar el backend
10. Paso 6 · La app React Native y el APK
11. Paso 7 · Verificación de punta a punta
12. Variaciones y otros usos
13. Seguridad al clonar
14. Solución de problemas
15. Referencias

01 Qué es y qué obtienes al clonarlo

CardioAlert es un sistema de monitoreo cardíaco en tiempo real con inteligencia artificial. Una banda **Polar H10** mide el electrocardiograma (ECG) del paciente, una app Android lo recibe por Bluetooth y un modelo de IA lo clasifica como **Normal**, **Fibrilación auricular** o **Isquemia miocárdica**, explicando qué parte de la señal motivó la decisión. Si hay una anomalía, el paramédico envía una alerta que el médico del hospital recibe al instante.

Al clonar el repositorio obtienes

PIEZA	QUÉ HACE	CARPETA
App móvil	App Android en React Native con módulo nativo para el Polar H10. Roles de paramédico, médico y administrador; 12 pantallas	<code>CardioAlertMobile/</code>
Backend	API REST (Node.js + Express) que guarda sesiones, diagnósticos, alertas y usuarios	<code>backend/</code>
API de IA	Servicio Python (FastAPI + TensorFlow) que clasifica el ECG y genera la explicación. Incluye el modelo ya entrenado	<code>ml-api/</code>
Base de datos	Script que crea tablas, reglas de seguridad por rol y avisos en tiempo real en Supabase	<code>supabase/</code>
Entrenamiento	Notebook de Google Colab que descarga los datos públicos y genera el modelo	<code>training/</code>
Despliegue alternativo	Dockerfiles y <code>docker-compose</code> para alojar los servicios en cualquier servidor	<code>deploy/</code>

Qué NO viene en el repositorio (y está bien que no venga)

Las llaves y contraseñas: cada quien que clona crea **sus propias** cuentas de Supabase y Railway y genera sus propias llaves. Los archivos `.env` están excluidos de Git; en su lugar hay plantillas `.env.example`. Clonar el repositorio **no** da acceso a la base de datos ni a los servidores del proyecto original.

Uso clínico

CardioAlert es un prototipo académico de soporte a la decisión, validado con datos públicos de PhysioNet. No es un dispositivo médico certificado y no reemplaza el criterio de un profesional de salud.

02 Arquitectura y dónde se aloja cada parte

Polar H10

hardware · pecho del paciente

Mide el ECG (130 Hz) y lo envía por Bluetooth LE

▼ Bluetooth LE

App CardioAlert

APK · celular Android 13+

Recibe y procesa la señal, muestra el diagnóstico y envía alertas

▼ HTTPS

Backend · API de IA

Railway (o alternativas)

El backend guarda los datos clínicos; la API de IA clasifica cada ventana de 10 s

▼ Llave secreta · solo desde el servidor

Supabase

PostgreSQL · Auth · Realtime

Base de datos con seguridad por rol, login y avisos en tiempo real al médico

Fuera de la operación diaria está **Google Colab**, donde se entrena el modelo una sola vez. El archivo resultante (`.keras`) se copia a la API de IA.

Opciones de alojamiento

PIEZA	OPCIÓN USADA	ALTERNATIVAS PROBADAS O COMPATIBLES
Base de datos	Supabase (nube, plan gratuito)	Supabase autoalojado con Docker
Backend	Railway	Render · cualquier servidor con Docker (VPS) · Google Cloud Run · Fly.io
API de IA	Railway	Las mismas; requiere ~1 GB de RAM por TensorFlow
Entrenamiento	Google Colab (GPU T4 gratuita)	Cualquier equipo con Python 3.12 y, idealmente, GPU
App	APK instalado a mano	Publicación en Google Play (requiere firma propia)

Los Dockerfiles de `deploy/` se construyeron y ejecutaron con Docker 29: ambos servicios arrancaron y la API de IA dio exactamente los mismos resultados que en Railway.

03 Requisitos y costos

Cuentas (todas con plan gratuito para empezar)

SERVICIO	PARA QUÉ
GitHub	Clonar el código y conectarlo a Railway
Supabase	Base de datos, login y tiempo real
Railway (u otra plataforma)	Alojar el backend y la API de IA
Google	Colab y Drive, solo si vas a reentrenar el modelo

Herramientas en tu computadora

HERRAMIENTA	VERSIÓN	PARA QUÉ	INSTALACIÓN EN MACOS
Git	cualquiera	Clonar el repositorio	<code>xcode-select --install</code>
Node.js	22 o superior	App y backend	<code>brew install node</code>
JDK	17	Compilar la app Android	<code>brew install openjdk@17</code>
Android SDK	plataforma 37, NDK 27.1	Compilar el APK	<code>brew install --cask android-commandlinetools</code>
Python	3.12 (no 3.13+)	Solo para correr la API de IA en local	<code>brew install python@3.12</code>
Docker	opcional	Solo para alojar con Docker	Docker Desktop

En Windows o Linux se usan los instaladores oficiales de cada herramienta. Para el SDK de Android, la vía más simple es instalar Android Studio, que lo incluye. No hace falta Android Studio para compilar; solo el SDK.

Hardware

- **Celular Android 13 o superior**, exigido por el SDK del Polar.
- **Banda Polar H10** (unos 90 USD). Es opcional: sin ella la app funciona con ECG simulado.

Costos aproximados

CONCEPTO	COSTO
Supabase	Gratis en el plan Free (suficiente para un piloto)

CONCEPTO	COSTO
Railway	Desde ~5 USD/mes (plan Hobby); el consumo depende del uso y de la RAM de TensorFlow
Google Colab	Gratis (GPU T4 con límite de uso)
Render (alternativa)	El plan gratuito tiene 512 MB de RAM, insuficiente para la API de IA; se necesita un plan pago

Los precios cambian: verifica los vigentes en el sitio de cada proveedor.

04 Estructura del repositorio

```

cardioalert/
├── CardioAlertMobile/           App React Native (TypeScript)
│   ├── android/                Proyecto Android nativo (Gradle) + módulo Kotlin del Polar
│   │   ├── src/                Pantallas, estado, procesamiento de señal, IA, datos
│   │   └── .env.example         Plantilla de configuración de la app
│   └── backend/                API Express (TypeScript)
│       ├── src/routes/         /sessions /alerts /users
│       └── .env.example
├── ml-api/                     API de IA (Python)
│   ├── main.py                 /predict /explain
│   ├── model/                  cardioalert_cnn_bilstm.keras (modelo entrenado)
│   ├── requirements.txt        .python-version · Procfile
├── supabase/schema.sql          Tablas, seguridad por rol (RLS) y tiempo real
├── training/                   Notebook de Colab para entrenar el modelo
├── deploy/                     Dockerfiles y docker-compose (alojamiento alternativo)
└── SETUP.md

```

05 Paso 1 · Clonar el repositorio

Hay dos formas, según lo que quieras hacer:

Fork (recomendado)

En GitHub, botón **Fork**. Crea una copia en tu cuenta que puedes conectar a Railway y modificar libremente.

Clonar directo

Solo para probar en tu computadora. Para desplegar necesitarás subirlo a un repositorio tuyo.

```

git clone https://github.com/TU_USUARIO/cardioalert.git
cd cardioalert

```

A partir de aquí, cada paso crea **tu propia** infraestructura. El orden importa: base de datos → modelo → API de IA → backend → app, porque cada pieza necesita la dirección de la anterior.

06 Paso 2 · Base de datos en Supabase

2.1 Crear el proyecto

- 1 En **supabase.com**: **New project**. Nombre (por ejemplo `cardioalert`) y la región más cercana a tus usuarios.
- 2 **Database password**: "Generate a password" y guárdala en un gestor de contraseñas. La app no la usa.
- 3 **Security**: deja marcadas *Enable Data API* y *Automatically expose new tables* (el schema protege todas las tablas con RLS), y marca *Enable automatic RLS* como red de seguridad.

2.2 Crear las tablas

- 1 **SQL Editor** → **New query**, pega el contenido completo de `supabase/schema.sql`.
- 2 Selecciona todo (`Cmd/Ctrl + A`) y ejecuta con **Run**. Debe decir "Success. No rows returned".
- 3 Verifica con esta consulta:

```
select
  (select count(*) from information_schema.tables where table_schema = 'public'
   and table_name in ('profiles','sessions','classifications','alerts')) as tablas,
  (select count(*) from pg_policies where schemaname = 'public') as politicas,
  (select count(*) from pg_trigger where tgname = 'on_auth_user_created') as trigger;
```

Resultado correcto: **tablas 4 · politicas 8 · trigger 1.**

2.3 Obtener las llaves

DATO	DÓNDE	VA EN
Project URL	Project Settings → Data API	App y backend
Publishable key (<code>sb_publishable_...</code>) o la legacy <code>anon</code>	Project Settings → API Keys	Solo la app
Secret key (<code>sb_secret_...</code>) o la legacy <code>service_role</code>	Project Settings → API Keys	Solo el backend

2.4 Crear el primer administrador

Por seguridad, las cuentas de administrador no se crean desde la app. El primero se crea en el panel:

- 1 **Authentication** → **Users** → **Add user** → **Create new user**, con tu correo y una contraseña robusta, y *Auto confirm user* marcado.

- 2 En el **SQL Editor**, asígnale el rol:

```
update public.profiles
set role = 'admin', full_name = 'Administrador'
where id = (select id from auth.users where email = 'admin@tu-dominio.com');
```

Desde ese momento, el administrador crea las cuentas de paramédicos y médicos dentro de la app.

07 Paso 3 · Modelo de IA en Google Colab

El repositorio ya incluye un modelo entrenado en `ml-api/model/cardioalert_cnn_bilstm.keras`.

Puedes saltarte este paso si te sirven las mismas tres clases. Reentrena si quieres mejorar el modelo o adaptarlo a otras clases (sección 12).

Qué hace el notebook

ETAPA	DETALLE
Datos	Descarga de PhysioNet: MIT-BIH Arrhythmia, MIT-BIH Atrial Fibrillation y European ST-T (~1.2 GB). Se guardan en tu Drive para no volver a descargarlos
Etiquetado	Ventanas de 10 s. Cada base aporta casos normales y patológicos, así la base de origen no delata la clase
Preprocesamiento	Idéntico al de la app: 250 Hz, filtro 0.5–40 Hz, z-score
División	Por paciente: la evaluación usa pacientes que el modelo nunca vio
Modelo	CNN-BiLSTM con aumento de datos (polaridad y ruido)
Resultado	Métricas de prueba, controles de atajo y el archivo <code>cardioalert_cnn_bilstm.keras</code>

Cómo ejecutarlo

- 1 En colab.research.google.com: **Archivo** → **Subir notebook** → `training/CardioAlert_entrenamiento_corregido.ipynb`.
- 2 **Entorno de ejecución** → **Cambiar tipo de entorno** → **T4 GPU**.
- 3 **Entorno de ejecución** → **Ejecutar todo**. Al inicio pide permiso para usar tu Drive.
- 4 Espera: la descarga tarda 30–90 min la primera vez y el entrenamiento 10–20 min. Si Colab se desconecta, vuelve a ejecutar: los datos ya están en Drive.
- 5 Al final se descarga el `.keras` y queda una copia en `Drive/proyecto_ecg/`, junto con sus métricas en JSON.

- Reemplaza el archivo en `ml-api/model/` con el mismo nombre y haz `git push`.

Prueba rápida

En la celda de configuración, `QUICK_TEST = True` usa solo 3 pacientes y 1 época: sirve para comprobar en minutos que todo funciona. Las métricas de esa corrida no significan nada.

Versiones

La API de IA usa TensorFlow 2.20. Si Colab trae una versión muy distinta y el modelo no carga en la API, alinea `tensorflow` en `ml-api/requirements.txt` con la versión que muestra la primera celda del notebook.

08 Paso 4 · Desplegar la API de IA

Inventa una clave larga para proteger la API (será la `ML_API_KEY`). En macOS o Linux puedes generarla con:

```
openssl rand -hex 24
```

Opción A · Railway (la usada en el proyecto)

- En [railway.app](#): **New Project** → **Deploy from GitHub repo** → tu fork.
- En el servicio → **Settings: Root Directory** `/ml-api` y, en **Build** → **Watch Paths**, `/ml-api/**`, para que solo se redespliegue cuando cambie esa carpeta.
- Variables:** `ML_API_KEY` = tu clave. **No agregues** `PORT`: Railway la asigna.
- Deploy.** La primera vez tarda 3–5 min porque instala TensorFlow. Railway detecta Python 3.12 por `.python-version` y el arranque por el `Procfile`.
- Settings** → **Networking** → **Generate Domain**, con puerto `8080`. Guarda la URL: es la `ML_API_URL` de la app.

Opción B · Render (sin Docker)

CAMPO	VALOR
Tipo	Web Service, desde tu repositorio
Root Directory	<code>ml-api</code>
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>uvicorn main:app --host 0.0.0.0 --port \$PORT</code>

CAMPO	VALOR
Variables	ML_API_KEY y PYTHON_VERSION=3.12.7
Plan	Uno con al menos 1 GB de RAM: el gratuito no alcanza para TensorFlow

Opción C · Docker (servidor propio, Google Cloud Run, Fly.io...)

Desde la raíz del repositorio:

```
docker build -f deploy/ml-api.Dockerfile -t cardioalert-ml-api ml-api
docker run -d -p 8000:8000 -e ML_API_KEY=tu-clave cardioalert-ml-api
```

La imagen respeta la variable `PORT`, así que funciona sin cambios en plataformas que la definen, como Cloud Run. En Cloud Run, sube la imagen a Artifact Registry y desplégala con al menos `--memory 2Gi`.

Probarla

```
curl https://TU-ML-API/health
# {"status":"ok","model_loaded":true,"classes":["normal","afib","isquemia"]}
```

09 Paso 5 · Desplegar el backend

Variables que necesita

VARIABLE	VALOR
SUPABASE_URL	Project URL de tu Supabase
SUPABASE_SERVICE_ROLE_KEY	La secret key de Supabase (acepta <code>sb_secret_...</code> o la legacy <code>service_role</code>)
PORT	Solo en local o Docker. En Railway y Render no se define

Opción A · Railway

- 1 En el **mismo proyecto** de Railway: **+ Create** → **GitHub Repo** → tu fork otra vez.
- 2 *Root Directory* `/backend` ; *Watch Paths* `/backend/**`.
- 3 **Variables**: las dos de la tabla. **Deploy**.
- 4 **Generate Domain** (puerto `8080`). Esa URL es la `API_BASE_URL` de la app.

Railway compila solo: `npm install`, `npm run build` (TypeScript) y `npm start`.

Opción B · Render

Web Service con *Root Directory* `backend`, *Build* `npm install && npm run build`, *Start* `npm start` y las dos variables. El plan gratuito sirve para el backend, pero se "duerme" tras un rato sin uso y la primera petición tarda cerca de un minuto.

Opción C · Todo en un servidor con Docker Compose

Levanta backend y API de IA juntos en un VPS (DigitalOcean, Hetzner, AWS Lightsail...) o en una computadora propia:

```
cp backend/.env.example backend/.env      # completa SUPABASE_URL y la secret key
cd deploy
cp .env.example .env                      # completa ML_API_KEY
docker compose up -d --build
docker compose ps                          # ambos servicios en "running"
```

HTTPS es obligatorio

El APK final de Android bloquea las conexiones `http://` sin cifrar. En un servidor propio pon delante un proxy con certificado. Con **Caddy**, que obtiene el certificado automáticamente, basta un `Caddyfile` así:

```
api.tu-dominio.com {
    reverse_proxy localhost:3000
}
ia.tu-dominio.com {
    reverse_proxy localhost:8000
}
```

Probarlo

```
curl https://TU-BACKEND/health            # {"ok":true}
curl -i https://TU-BACKEND/sessions        # 401: exige login, es lo correcto
```

10 Paso 6 · La app React Native y el APK

Cómo está hecha

Es un proyecto **React Native "bare"**, sin Expo: la interfaz y la lógica están en TypeScript, y dentro de `android/` hay un proyecto Android normal con un **módulo nativo en Kotlin** que usa el SDK oficial de Polar. No se usa Expo porque su modo administrado no permite incluir ese SDK nativo. La configuración se lee de `.env` con `react-native-dotenv` y queda incrustada al compilar.

6.1 Configurar

```
cd CardioAlertMobile
npm install
cp .env.example .env
```

VARIABLE	VALOR
<code>SUPABASE_URL</code>	Project URL de tu Supabase
<code>SUPABASE_ANON_KEY</code>	La publishable (o <code>anon</code>). Nunca la secret
<code>API_BASE_URL</code>	URL de tu backend, con <code>https://</code>
<code>ML_API_URL</code> · <code>ML_API_KEY</code>	URL y clave de tu API de IA. Si quedan vacías, la app usa el clasificador de simulación
<code>ALERT_CONFIDENCE_THRESHOLD</code>	Confianza mínima para disparar una alerta (<code>0.85</code>)

Si `SUPABASE_URL` se deja con el valor de ejemplo, la app arranca en **modo demo**: guarda todo en el celular y acepta cualquier login. Sirve para ver la interfaz sin montar nada.

6.2 Preparar el SDK de Android

Una sola vez. En macOS con Homebrew:

```
export JAVA_HOME=/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home
export ANDROID_HOME=/opt/homebrew/share/android-commandlinetools
yes | sdkmanager --licenses
sdkmanager "platform-tools" "platforms;android-37.0" "build-tools;37.0.0"
"ndk;27.1.12297006"
echo "sdk.dir=$ANDROID_HOME" > android/local.properties
```

En Windows y Linux, apunta `JAVA_HOME` al JDK 17 y `ANDROID_HOME` a la carpeta del SDK (la de Android Studio, si lo instalaste), e instala los mismos paquetes con `sdkmanager`.

6.3 Generar el APK

```
cd android
./gradlew assembleRelease -PreactNativeArchitectures=arm64-v8a # Windows: gradlew.bat
```

- La primera vez tarda varios minutos: descarga Gradle, las librerías y el Polar SDK desde JitPack.
- Resultado: `android/app/build/outputs/apk/release/app-release.apk` (~36 MB).
- `arm64-v8a` limita el APK a los procesadores de los celulares actuales. Sin ese parámetro sirve para cualquier procesador, pero pesa ~80 MB.
- **Cada cambio en `.env` exige volver a generar el APK.**

6.4 Desarrollo en vivo (opcional)

Con un celular conectado por USB y la depuración USB activada, los cambios de código se ven al instante:

```
npm start           # terminal 1: Metro, el servidor de desarrollo
npm run android     # terminal 2: instala la versión de desarrollo
```

6.5 Instalar y distribuir

- Envía el APK al celular (WhatsApp, Drive, cable), ábrelo y permite "instalar apps de origen desconocido".
- El APK se firma con la llave de depuración pública de React Native: sirve para pruebas y demos. **Para Google Play** genera tu propia llave con `keytool` y configúrala en `signingConfigs` de `android/app/build.gradle`, siguiendo la guía oficial "Publishing to Google Play Store" de React Native.

11 Paso 7 • Verificación de punta a punta

COMPROBACIÓN	CÓMO	RESULTADO ESPERADO
Base de datos	Consulta del paso 2.2	4 tablas · 8 políticas · 1 trigger
API de IA	<code>curl .../health</code>	<code>model_loaded: true</code>
API de IA protegida	<code>POST /predict</code> sin <code>X-API-Key</code>	401
Backend	<code>curl .../health</code>	<code>{"ok":true}</code>
Backend protegido	<code>curl .../sessions</code> sin login	401
Login admin	Entrar en la app con rol Admin	Abre "Administración" y lista los usuarios
Crear usuarios	Crear un paramédico y un médico desde la app	"Cuenta creada" y aparecen en la lista
Sensor	Paramédico → Conectar → Buscar Polar H10	El sensor aparece con su ID y conecta
Diagnóstico	Iniciar Monitoreo	A los 10 s aparece el diagnóstico con la etiqueta "MODELO IA"
Alerta en tiempo real	Perfil → Simulación → Isquemia → Enviar alerta; el médico en otro celular	La alerta le aparece al médico al instante

12 Variaciones y otros usos

El sistema está separado en piezas con contratos claros, así que se puede adaptar cambiando pocas partes. Lo más común:

QUIERO...	QUÉ CAMBIAR
Otro nombre de app	<code>app_name</code> en <code>android/app/src/main/res/values/strings.xml</code> . Hoy dice "CardioAlertMobile"
Instalarla junto a la original	<code>applicationId</code> en <code>android/app/build.gradle</code> (por ejemplo <code>com.tuorg.monitor</code>)
Otro ícono	Las imágenes de <code>android/app/src/main/res/mipmap-*</code>
Cambiar el hospital receptor o el umbral	<code>RECEIVING_HOSPITAL</code> en <code>src/config.ts</code> y <code>ALERT_CONFIDENCE_THRESHOLD</code> en <code>.env</code>
Otras patologías o clases	Tres lugares con la misma lista y el mismo orden: <code>CLASSES</code> del notebook, <code>CLASSES</code> de <code>ml-api/main.py</code> y <code>LABELS</code> de <code>src/ml/types.ts</code> (más sus nombres y colores en <code>src/ui/theme.ts</code>). Además, el tipo <code>classification_type</code> de <code>schema.sql</code> y los esquemas Zod del backend
Otro modelo de IA	Cualquier modelo Keras que reciba <code>(2500, 1)</code> y devuelva una probabilidad por clase: reemplaza el <code>.keras</code> . La app no cambia
Otra frecuencia o duración de ventana	<code>TARGET_SAMPLE_RATE</code> y <code>WINDOW_SECONDS</code> en <code>src/signal/preprocess.ts</code> , en el notebook con los mismos valores, y el tamaño esperado en <code>ml-api/main.py</code>
IA sin internet (en el celular)	Exportar el modelo a TensorFlow Lite y usar <code>react-native-fast-tflite</code> . El punto de entrada ya está preparado en <code>src/ml/classifier.ts</code>
Otro sensor Polar (Verity Sense, OH1...)	El mismo Polar SDK los soporta; se ajustan las funciones de streaming del módulo Kotlin (por ejemplo PPG en lugar de ECG)
Un sensor de otra marca	Un módulo nativo nuevo con el SDK de ese fabricante que emita los mismos eventos (<code>PolarEcgData</code> , etc.). El resto de la app no cambia
iPhone	Portar <code>PolarModule.kt</code> a Swift con la versión iOS del Polar SDK. Sin ese módulo, la app en iOS funciona en modo simulado
Otro uso clínico (sin IA)	Dejar <code>ML_API_URL</code> vacío y aprovechar el monitoreo en vivo, el historial y las alertas
Base de datos propia (sin Supabase nube)	Supabase ofrece una versión autoalojada con Docker, compatible con el mismo <code>schema.sql</code>

13 Seguridad al clonar

- **Genera tus propias llaves.** Nunca reutilices llaves del proyecto original ni de otra persona.

- **La secret key de Supabase solo va en el servidor** (variables de Railway o `backend/.env`). Con ella se salta toda la seguridad de la base de datos. Nunca en la app, en Git ni en documentos compartidos.
- **No subas archivos `.env`**. El `.gitignore` ya los excluye; revisa con `git status` antes de cada commit.
- **La clave de la API de IA va dentro del APK**: frena el uso casual, pero alguien con conocimientos puede extraerla. Para un uso real, la API debería validar el login de Supabase igual que el backend.
- **Las cuentas de administrador** se crean solo desde el panel de Supabase; desde la app solo se crean paramédicos y médicos.
- **Datos de pacientes**: la app usa códigos anónimos (`PAC-1234`). Para datos reales, revisa la normativa local de protección de datos de salud.
- **Si una llave se filtra**: róta la de inmediato en el panel del proveedor y actualiza las variables en tus servidores.

14 Solución de problemas

SÍNTOMA	CAUSA PROBABLE	SOLUCIÓN
Railway responde "Application not found"	El despliegue aún no termina o falló	Espera; si persiste, revisa <i>Deployments</i> → <i>View logs</i>
Un cambio no se refleja en Railway	El despliegue nuevo falló y Railway mantiene el anterior	Revisa los logs del último despliegue
La API de IA se reinicia o cae	Falta de RAM para TensorFlow	Plan con 1 GB de RAM o más
Error instalando TensorFlow	Python 3.13 o superior	Usa Python 3.12 (<code>.python-version</code> o <code>PYTHON_VERSION</code>)
El dominio responde con error aunque el servicio corre	Puerto del dominio distinto al de la app	No defines <code>PORT</code> a mano en Railway; el dominio va al 8080
Gradle no encuentra <code>polar-ble-sdk</code>	Falta el repositorio JitPack o la versión no existe	Revisa <code>android/build.gradle</code> y la versión en <code>app/build.gradle</code>
"SDK location not found"	Falta <code>android/local.properties</code>	Crea el archivo con <code>sdk.dir=...</code> (paso 6.2)
La app ignora el <code>.env</code> nuevo	El APK se generó antes del cambio	Vuelve a generar el APK
La app no conecta con el backend	URL con <code>http://</code> en el APK final	Usa <code>https://</code>
"Esta cuenta no tiene el rol..." al iniciar sesión	El rol elegido en el login no coincide con el de la cuenta	Elige el rol correcto, o corrige la columna <code>role</code> del usuario en <code>profiles</code>

SÍNTOMA	CAUSA PROBABLE	SOLUCIÓN
Las consultas devuelven 0 filas	RLS: sin sesión o con otro rol no se ven datos	Es lo esperado; los datos se leen a través del backend
El APK no instala	Android anterior a la 13	Usa un celular con Android 13 o superior
<code>docker compose</code> : puerto ocupado	Otro programa usa el 3000 o el 8000	Cambia <code>BACKEND_PORT</code> y <code>ML_API_PORT</code> en <code>deploy/.env</code>

15 Referencias

- **Repositorio del proyecto:** github.com/ttiziu/cardioalert
- **Polar BLE SDK:** github.com/polarofficial/polar-ble-sdk
- **React Native**, entorno de desarrollo, módulos nativos y publicación en Google Play: reactnative.dev/docs
- **Supabase**, Auth, Row Level Security, Realtime y autoalojamiento: supabase.com/docs
- **Railway**, monorepos, Root Directory y Watch Paths: docs.railway.com
- **Render:** render.com/docs · **Google Cloud Run:** cloud.google.com/run/docs · **Caddy:** caddyserver.com/docs
- **PhysioNet**, bases de datos de ECG: physionet.org